



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA GRUPAL SEMANA 09

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web Fullstack

Tema: Lenguaje Python: Sintaxis, Estructuras de Control, POO (Clases, Herencia, Polimorfismo, Encapsulamiento), Excepciones y Módulos

Modalidad: Presencial / Laboratorio

Fecha: 11.06.2026

Duración: 60 minutos

APELLIDOS Y NOMBRES:

- BARJA ORTIZ, ERICK
- NAVARRO SERVA, LESLY
- SULLUCHUCO VILCAPOMA, ANYELO
- TORIBIO ANSELMO, DAVID
- YAURI TORRES, BENJAMIN

Consigna Técnica

Formen equipos 6 estudiantes. Deben implementar un **Sistema de Gestión de Productos (CLI)** que cumpla:

1. Arquitectura POO: Clase base Producto, clases derivadas Electronico y Libro. Herencia múltiple opcional (DescuentoMixin).
2. Encapsulamiento: Atributos privados/protegidos, acceso vía @property, validación de rangos en setters.
3. Polimorfismo: Método calcular_precio_final() sobrescrito por tipo. __str__ para representación uniforme.
4. Excepciones: ValidationError, NotFoundError. Bloques try/except/else/finally en flujo de usuario.
5. Modularización: main.py (orquestador), models/ (clases), utils/ (validaciones, formateo), __init__.py en cada paquete.

Roles Técnicos

Rol	Responsabilidad
Arquitecto POO	Diagrama UML, jerarquía de clases, MRO, super(), __init__/_del__



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Desarrollador de Lógica	Funciones imperativas, estructuras de control, diccionarios/listas, flujo CLI
Ingeniero de Excepciones & Validación	try/except, excepciones custom, @property, sanitización de entrada
QA & Documentador	Pylint/Black, pruebas de ruta feliz/error, README.md, justificación técnica

Entregable

- Enlace del Repositorio GitHub o ZIP con: main.py, models/, utils/, README.md, requirements.txt (vacío o con python>=3.11)
- README.md debe incluir:
 - Diagrama de clases (herencia, atributos, métodos)
 - Explicación de MRO aplicado y por qué se eligió herencia vs composición
 - Capturas de ejecución (caso exitoso + 2 excepciones capturadas)

Rúbrica de Evaluación (Laboratorio 09A)

Criterio	Peso
Diseño POO correcto (clases, atributos, métodos, super())	25%
Encapsulamiento (@property) y polimorfismo funcional	20%
Manejo robusto de excepciones (try/except/else/finally)	20%
Estructuras de control, funciones imperativas y tipos compuestos	15%
Modularización, PEP 8, documentación y trabajo en equipo	20%



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar) main.py

python

```
1  #!/usr/bin/env python3
2  """Orquestador CLI del Sistema de Gestión de Productos - IS093A Semana 10"""
3
4  from models.producto import Producto, Electronico, Libro
5  from utils.validators import validar_numero, validar_categoria
6  from utils.exceptions import ValidationError, NotFoundError
7
8  def menu():
9      print("\n=== GESTIÓN DE PRODUCTOS (CLI) ===")
10     print("1. Agregar Producto")
11     print("2. Listar Productos")
12     print("3. Calcular Precio Final")
13     print("4. Salir")
14     return input("Opción > ").strip()
15
16  def main():
17     inventario = {} # {id: objeto_producto}
18     contador_id = 1
19
20     while True:
21         try:
22             opcion = menu()
23             if opcion == "1":
24                 # TODO: Solicitar datos, validar con utils.validators
25                 # Crear instancia según categoría, asignar ID, guardar en inventario
26                 pass
27             elif opcion == "2":
28                 # TODO: Iterar inventario, manejar caso vacío
29                 pass
30             elif opcion == "3":
31                 # TODO: Buscar producto por ID, llamar calcular_precio_final()
32                 # Capturar NotFoundError y ValidationError
33                 pass
34             elif opcion == "4":
35                 print("Cerrando sistema...")
36                 break
37             else:
38                 raise ValidationError("Opción inválida")
39         except ValidationError as e:
40             print(f"⚠ Error de validación: {e}")
41         except NotFoundError as e:
42             print(f"🔍 No encontrado: {e}")
43         except Exception as e:
44             print(f"❌ Error inesperado: {e}")
45         finally:
46             # TODO: Log o cleanup opcional
47             pass
48
49  if __name__ == "__main__":
50     main()
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



models/__init__.py

python

```
1 # TODO: Exponer clases públicas para importación Limpia
2 # from .producto import Producto, Electronico, Libro
```

models/producto.py

python

```
1 from __future__ import annotations
2 from typing import override
3
4 class Producto:
5     def __init__(self, nombre: str, precio_base: float, stock: int):
6         # TODO: Encapsular atributos, validar rangos, asignar con @property
7         self.nombre = nombre
8         self.precio_base = precio_base
9         self.stock = stock
10
11     def calcular_precio_final(self) -> float:
12         raise NotImplementedError("Subclases deben implementar este método")
13
14     def __str__(self) -> str:
15         return f"[{self.__class__.__name__}] {self.nombre} | S/ {self.precio_base} | Stock: {self.stock}"
16
17     def __del__(self):
18         print(f"🗑 Destruyendo instancia: {self.nombre}")
19
20 class Electronico(Producto):
21     def __init__(self, nombre: str, precio_base: float, stock: int, garantia_meses: int):
22         # TODO: Llamar super(), validar garantía, atributo protegido
23         pass
24
25     @override
26     def calcular_precio_final(self) -> float:
27         # TODO: Aplicar IGV (18%) + descuento si stock > 50
28         pass
29
30 class Libro(Producto):
31     def __init__(self, nombre: str, precio_base: float, stock: int, autor: str):
32         # TODO: Llamar super(), validar autor, atributo privado __autor
33         pass
34
35     @override
36     def calcular_precio_final(self) -> float:
37         # TODO: Precio base - 10% si es libro de texto
38         pass
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



utils/exceptions.py

python

```
1 # TODO: Definir excepciones personalizadas
2 # class ValidationError(Exception): ...
3 # class NotFoundError(Exception): ...
```

utils/validators.py

python

```
1 from .exceptions import ValidationError
2
3 def validar_numero(valor: str, tipo: type = float, min_val: float = 0, max_val: float = None) -> float:
4     try:
5         num = tipo(valor)
6         if num < min_val or (max_val is not None and num > max_val):
7             raise ValidationError(f"valor fuera de rango: {min_val}-{max_val}")
8         return num
9     except ValueError:
10        raise ValidationError(f"Formato numérico inválido: {valor}")
11
12 def validar_categoria(categoria: str) -> str:
13     # TODO: Validar contra ["electronico", "Libro"], normalizar, lanzar ValidationError
14     pass
```

NOTAS TÉCNICAS CLAVE

Concepto	Aplicación Práctica
Indentación & PEP 8	Python usa 4 espacios. No mezclar tabs/espacios. black formatea automáticamente. pylint detecta antipatronos.
Encapsulamiento	<code>_attr</code> = protegido (convenio), <code>__attr</code> = name-mangling (privado). <code>@property</code> valida en asignación sin romper API.
Herencia Múltiple & MRO	Python usa C3 linearization. <code>super()</code> sigue el orden de resolución, no solo la clase padre directa. Evitar diamantes sin diseño explícito.
Constructores/Destructores	<code>__init__</code> inicializa estado. <code>__del__</code> se llama al garbage collection (no determinista). Usar <code>contextlib</code> o <code>try/finally</code> para cleanup seguro.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



Polimorfismo	Python es dinámico: no requiere interfaces explícitas. Si un objeto tiene el método, se ejecuta (duck typing). @override (Python 3.12+) ayuda a detectar errores.
---------------------	---

FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

Excepciones	try/except/else/finally: else si no hay error, finally siempre. Nunca capturar Exception genérico sin relanzar o log. Crear jerarquía custom para dominio.
--------------------	--

QA & DOCUMENTADOR:

INTRODUCIR EL CODE pip install pylint black, PARA VERIFICAR

```
PS C:\Users\LESLY\Documents\GitHub\semana10> pip install pylint black
Collecting pylint
  Downloading pylint-4.0.5-py3-none-any.whl.metadata (12 kB)
Collecting black
  Downloading black-26.5.1-cp312-cp312-win_amd64.whl.metadata (95 kB)
    95.1/95.1 kB 601.9 kB/s eta 0:00:00
Collecting astroid<=4.1.dev0,>=4.0.2 (from pylint)
  Downloading astroid-4.0.4-py3-none-any.whl.metadata (4.4 kB)
Requirement already satisfied: colorama>=0.4.5 in c:\users\lesly\appdata\roaming\python\python312\site-packages (from pylint) (0.4.6)
Collecting dill>=0.3.6 (from pylint)
  Downloading dill-0.4.1-py3-none-any.whl.metadata (10 kB)
Collecting isort!=5.13,<9,>=5 (from pylint)
  Downloading isort-8.0.1-py3-none-any.whl.metadata (11 kB)
```

INTRODUCIR EL CODE black main.py models/ utils/

```
PS C:\Users\LESLY\Documents\GitHub\semana10> black main.py models/ utils/
reformatted C:\Users\LESLY\Documents\GitHub\semana10\utils\exceptions.py
reformatted C:\Users\LESLY\Documents\GitHub\semana10\utils\validators.py
reformatted C:\Users\LESLY\Documents\GitHub\semana10\models\producto.py
reformatted main.py

All done! 🌟🍰🌟
4 files reformatted, 2 files left unchanged.
```

INTRODUCIR EL CODE pylint main.py models/ utils/

```
PS C:\Users\LESLY\Documents\GitHub\semana10> pylint main.py models/ utils/
***** Module main
main.py:136:0: C0301: Line too long (105/100) (line-too-long)
main.py:1:0: C0114: Missing module docstring (missing-module-docstring)
main.py:147:0: C0116: Missing function or method docstring (missing-function-docstring)
main.py:195:16: W1309: Using an f-string that does not have any interpolated variables (f-string-without-interpolation)
main.py:147:0: R0912: Too many branches (13/12) (too-many-branches)
***** Module models.producto
models\producto.py:86:0: C0301: Line too long (140/100) (line-too-long)
models\producto.py:133:0: C0301: Line too long (159/100) (line-too-long)
models\producto.py:171:0: C0301: Line too long (141/100) (line-too-long)
models\producto.py:1:0: C0114: Missing module docstring (missing-module-docstring)
```




UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



```
C:\INGENIERIA DE SISTEMAS\SEMESTRE IX\DESARROLLO APLICACIONES WEB\PRACTICA\Semana 10\semana10>py main.py
=====
SISTEMA DE GESTIÓN DE PRODUCTOS - CLI PYTHON
UNCP - Desarrollo de Aplicaciones Web
=====

MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 1
```

Agregar producto

```
.....
MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 1

--- AGREGAR PRODUCTO ---
Tipo de producto (1: Electrónico, 2: Libro): 1
Ingrese el ID del producto (entero): 3
Ingrese el nombre del producto: TABLET
Ingrese el precio base: S/. 500
Ingrese el porcentaje de descuento (0-100) [Por defecto 0]: 10
Ingrese los meses de garantía (entero mayor a 0): 3

[Éxito] Producto agregado al inventario: TABLET
✅ Operación ejecutada exitosamente sin interrupciones.
```

Listar producto

```
MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 2

--- INVENTARIO DE PRODUCTOS ---
[Electronico] ID: 1 | Nombre: LAPTOP | Precio Base: S/. 10.00 | Garantía: 1 meses | Descuento: 10.0% | Precio Final: S/. 9.00
[Electronico] ID: 2 | Nombre: COMPUTADORA | Precio Base: S/. 200.00 | Garantía: 1 meses | Descuento: 10.0% | Precio Final: S/. 180.00
[Electronico] ID: 3 | Nombre: TABLET | Precio Base: S/. 500.00 | Garantía: 3 meses | Descuento: 10.0% | Precio Final: S/. 450.00
[Libro] ID: 4 | Nombre: LIBRO | Precio Base: S/. 10.00 | Autor: Navarro Keny | Descuento: 0.0% | Precio Final: S/. 10.00

✅ Operación ejecutada exitosamente sin interrupciones.
```

Calcular totales



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



```
MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 3

--- CÁLCULO DE TOTALES ---

Desglose de Precios:
- LAPTOP (ID: 1): Base S/. 10.00 | Final S/. 9.00
- COMPUTADORA (ID: 2): Base S/. 200.00 | Final S/. 180.00
- TABLET (ID: 3): Base S/. 500.00 | Final S/. 450.00
- LIBRO (ID: 4): Base S/. 10.00 | Final S/. 10.00
-----
Total Base Acumulado:      S/. 720.00
Total Final Acumulado:     S/. 649.00
Ahorro Total Aplicado:     S/. 71.00
-----

✅ Operación ejecutada exitosamente sin interrupciones.
```

Eliminar producto

```
.....

MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 4

--- ELIMINAR PRODUCTO ---
Ingrese el ID del producto a eliminar: 3

[Destructor POO] Liberando recursos de memoria para ID: 3

[Éxito] Producto 'TABLET' (ID: 3) removido del inventario.

✅ Operación ejecutada exitosamente sin interrupciones.
```

Ver diagrama de resolución de métodos



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



```
.....
MENÚ DE OPCIONES:
1. Agregar Producto
2. Listar Productos (Polimorfismo __str__)
3. Calcular Totales (Polimorfismo calcular_precio_final)
4. Eliminar Producto (Demostración de Destructor __del__)
5. Ver Diagrama de Resolución de Métodos (MRO)
6. Salir

Seleccione una opción (1-6): 5

=====
ORDEN DE RESOLUCIÓN DE MÉTODOS (MRO)
=====

MRO para la clase Electronico:
1. Electronico
2. DescuentoMixin
3. Producto
4. object

MRO para la clase Libro:
1. Libro
2. DescuentoMixin
3. Producto
4. object

=====

✅ Operación ejecutada exitosamente sin interrupciones.
```

LINK DEL REPOSITORIO:

<https://github.com/IngenieroTaipe/semana10.git>